16

Domain-Specific RL: Code, Math, Tools, Dialogue

Domain-Specific Rewards: The Key to Specialization

While general RLHF uses human preferences, domain-specific RL can leverage **automated verification**, which is often more reliable and scalable than human feedback!

The pattern: (1) Identify domain with objective correctness metric, (2) use automated verifier as reward signal, (3) apply RL to optimize for verified correctness, (4) result: superhuman performance in narrow domain.

This approach has been extraordinarily successful for code generation (execution-based rewards), mathematics (formal proof verification), tool use (API success/failure), and multi-turn dialogue (conversation quality metrics).

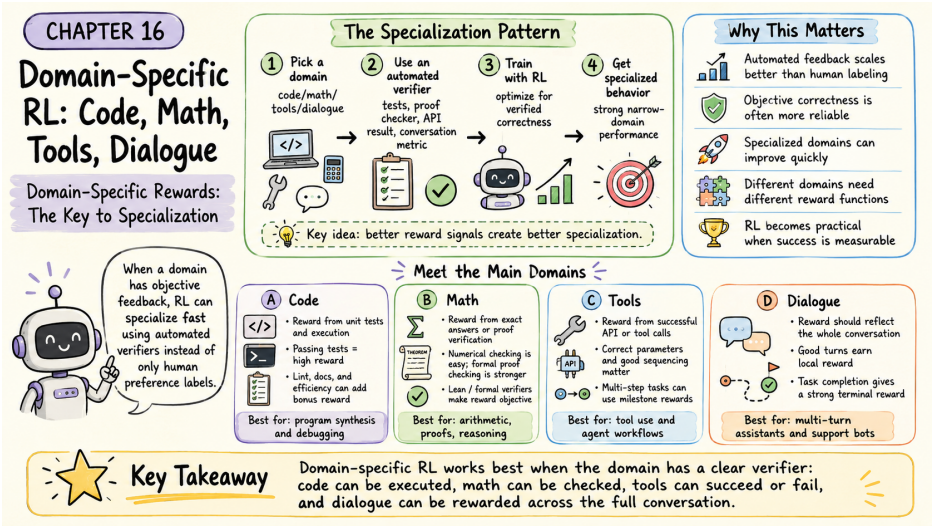

Figure 16.1: Domain-specific RL at a glance – when code, math, tools, and dialogue each provide their own automated verifier as the reward signal.

Code: Execution Feedback as Reward

Why Code Generation is Perfect for RL:

Unlike creative writing or conversation, code has an *objective measure of correctness*: does it run? Does it pass tests? This gives us a perfect reward signal: pass all tests → high reward (+10), pass some tests → partial reward (+1 to +5), syntax error → low reward (−2), runtime error → negative reward (−5), pass no tests → very negative reward (−10). No human labeling needed! The Python interpreter tells us if code works.

Formal Reward Function for Code:

For a code generation task with test suite $\mathcal{T} = \{t_1, \ldots, t_n\}$:

$$r_{\text{code}}(x, y) = \sum_{i=1}^{n} [\text{test}_i \text{ passes}] + 0.5 \cdot [\text{no lint errors}] + 0.3 \cdot [\text{has docstring}] + 0.2 \cdot [\text{efficient ex}$$

Evaluation Process: Model generates code solution, execute with all test inputs, count passing tests, check for bonus conditions (clean style, documentation, efficiency), sum all points for total reward.

Concrete Example: Problem: "Write function to reverse a string." Generated solution uses Python slicing `s[::-1]`, has docstring, passes linter, passes all 3 tests. Reward: $3 + 0.5 + 0.3 + 0.2 = 4.0$.

Training Loop for Code Generation

Practical RL Training for Code Generation:

Algorithm: For each training iteration, sample $n = 1000$ programming problems. For each problem: (a) generate a group of $k = 4$ code solutions, (b) execute each with test cases and compute reward (base reward from tests passed, bonuses for style/docs, penalty of -5 for syntax/runtime errors), (c) calculate advantages GRPO-style (each solution's reward minus group average), (d) update policy only on solutions better than group average.

Typical Training Results:

Iteration	pass@1	pass@10
0 (Base model)	35%	58%
1	42%	65%
2	51%	73%
3	59%	79%
5	68%	85%
10	75%	90%

After 10 iterations of RL training, the model more than **doubles** its success rate (35% → 75%)!

Math: Formal Verifiers as Reward

Two Types of Math Verification:

1. Numerical Verification (Easier). For arithmetic and numerical problems: model generates numerical answer, compare to ground truth, binary reward (correct = +1, incorrect = 0).

2. Formal Proof Verification (Harder). For proofs and formal mathematics: model generates proof in formal language (Lean, Coq, Isabelle), proof checker verifies logical validity, reward based on proof correctness. This is particularly exciting because it enables models to do *real mathematics*, not just arithmetic!

Formal Mathematics with Lean

In July 2024, Google DeepMind's AlphaProof (using RL with Lean theorem prover) solved 4 out of 6 problems from the International Mathematical Olympiad, earning a silver medal performance! The model generates candidate proofs, the Lean verifier checks validity, valid proofs earn high reward, and RL optimizes the policy to generate valid proofs. This was the first time AI reached medal-level performance on IMO–the key was using *formal verification* as the reward signal, completely objective with no human judgment needed.

Formal Verification Reward:

For theorem θ and proof π:

$$r_{\text{proof}}(\theta, \pi) = \begin{cases} +10 & \text{if Verify}(\theta, \pi) = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

Partial Credit Variant: For proof with n steps: $r_{\text{proof}}(\theta, \pi) = \sum_{i=1}^{n} [\text{step}_i \text{ is valid}]$. This rewards partially correct proofs, providing richer learning signal during training.

Verification Process: State theorem in formal language, model generates proof (sequence of logical steps), automated proof checker validates each step (Does this follow from previous steps? Are inference rules correctly applied? Does the conclusion match the goal?). If all steps valid → proof accepted → high reward. If any step invalid → proof rejected → no reward.

Tool Use: API Success as Reward

Tool-Using AI:

Modern AI systems don't just generate text–they use tools: call APIs (weather, search, calculator), execute code (Python interpreter), query databases (SQL), control applications (browser, email).

Each tool interaction gives immediate feedback: API call succeeds → good reward, API call fails → bad reward, result solves user problem → bonus reward. This enables models to learn effective tool use through RL!

Tool Use RL Example:

User Query: "What's the weather like in Tokyo right now?"

Attempt 1 (Novice Model):

Calls weather API with plain text string "Tokyo weather now." Result: ERROR–invalid parameter format. Reward: −5.

Attempt 2 (Learning):

Calls weather API with structured parameters: {city: "Tokyo", country: "Japan"}. Result: success, returns temp 22, condition "Sunny," humidity 65. Generated response: "The temperature in Tokyo is 22C with sunny skies." Total reward: +8.

Attempt 3 (Expert Model):

Calls weather API with comprehensive parameters: {city: "Tokyo", country: "JP", units: "metric"}. Result: success with full data. Generated response: "The current weather in Tokyo is 22C and sunny, with 65% humidity and light winds at 3.5 m/s." Total reward: +10.

Through RL training, the model learns to format API parameters correctly, select appropriate tools, present results comprehensively, and handle errors gracefully.

Multi-Tool Orchestration

Complex Tool Use: Chaining Multiple Tools

Real tasks often require multiple tools in sequence. Example: "Book me a flight to Paris next month" requires Calendar API (check availability), Flight search API (find flights), Booking API (reserve flight), Email API (send confirmation).

RL reward structure:

Milestone	Reward
Check calendar successfully	+1
Identify available dates	+2
Search flights for right dates	+2
Find flights in budget	+2
Successfully book	+5
Send confirmation	+1
Complete task correctly	+7
Maximum total	**+20**

Penalties: Wrong API parameters (−2), skip necessary step (−5), book wrong dates (−10), exceed budget (−15).

The model learns to plan multi-step actions, check preconditions, handle dependencies, and recover from errors.

Multi-Turn Dialogue: Conversation Quality

Most RLHF focuses on single-turn interactions. But real conversations have multiple turns! The challenge: reward must consider the entire conversation, early responses affect later ones (long-term consequences), and there's a credit assignment problem (which response was good/bad?).

Formal Multi-Turn Reward:

For conversation $C = \{(u_1, r_1), \ldots, (u_T, r_T)\}$:

Terminal reward: $R_{\text{term}}(C) = +10$ if task completed, +5 for partial progress, 0 for no progress, −5 if task failed.

Per-turn rewards: $r_t = r_{\text{quality}}(r_t) + r_{\text{progress}}(C_{1:t})$.

Total reward: $R_{\text{total}}(C) = \sum_{t=1}^{T} \gamma^{T-t} r_t + R_{\text{term}}(C)$, where γ (typically 0.95–0.99) is the discount factor. Earlier turns get discounted more (focus on recent quality).

In plain English: Each turn gets a small reward for local quality and progress toward the goal. At the end, a large terminal reward if the conversation succeeded.

Multi-Turn RL Example: Customer Support

Scenario: User needs help with password reset.

Conversation 1 (Poor):

Turn	Exchange	Reward
1	User: "I can't log in" / Bot: "Have you tried Incognito mode?"	+1
2	User: "That doesn't help" / Bot: "Clear your cookies"	+1
3	User: "Still can't log in!" / Bot: "Try a different browser"	0
4	User: "This isn't working. Bye." (Task failed)	−5
	Total reward:	−3

Bot never diagnosed the actual problem (forgotten password)!

Conversation 2 (Good):

Turn	Exchange	Reward
1	User: "I can't log in" / Bot: "Are you getting an error message?"	+2
2	User: "It says 'incorrect password' " / Bot: "Let me help reset your password…"	+3
3	User: "Got the reset email, thanks!" / Bot: "Great! Let me know if you need anything else." (Task completed)	+12
	Total reward:	**+17**

What the model learns: Diagnostic questions early lead to higher terminal reward. Random suggestions without diagnosis yield low reward. Quick resolution earns bonus reward. User satisfaction signals ("thanks!") provide positive signal.

Companion notebook. The code for this chapter is in `code/notebooks/part3_advanced/16_` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.